

How to Design Our Organization and Architecture with Conway's Law

From the previous session, we already learned [how we organize our teams affects how we perform our work](#). Dr. Melvin Conway's famous 1968 experiment led to what is now known as [Conway's Law](#).

Organizations tend to design systems that mirror how their teams communicate with each other. The larger the organization, the less flexible it becomes, and the stronger this effect is.

Eric S. Raymond simplified this: ["If you have four teams building a compiler, you will end up with a 4-stage compiler."](#)

To achieve fast and high-quality delivery from Development to Operations, [we need to organize our teams and work in a way that makes Conway's Law work to our advantage](#).

DevOps is about breaking down the wall between developers and operations so software can be delivered faster and more reliably (Here, "Ops" means platform operations—keeping the systems and infrastructure running—not general business operations).

The Etsy Sprouter Story: An example of how Conway's Law can either delay or reinforce our goals

Sprouter: The Middleware Layer Between PHP and Postgres

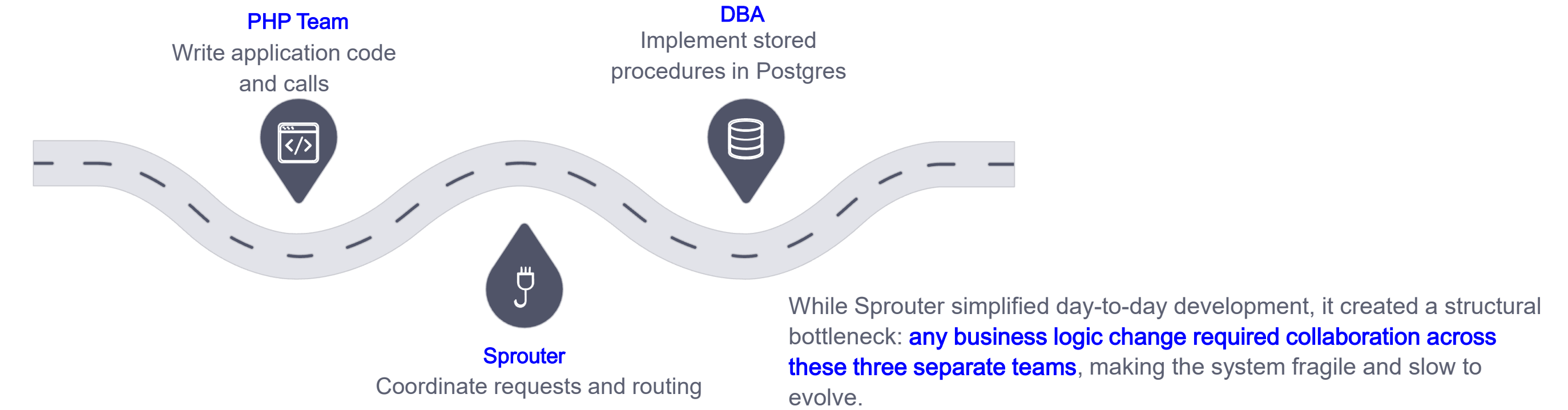
Sprouter was designed so that development teams could write code in PHP while database administrators wrote SQL in Postgres — with Sprouter acting as the middleware layer coordinating all interactions between the application and the database. Developers never had to touch database internals directly.

What Sprouter Did

Hid all database implementation details. Developers only worked with a simple interface, focusing purely on business logic — not on how data was stored or how queries ran.

The Hidden Problem

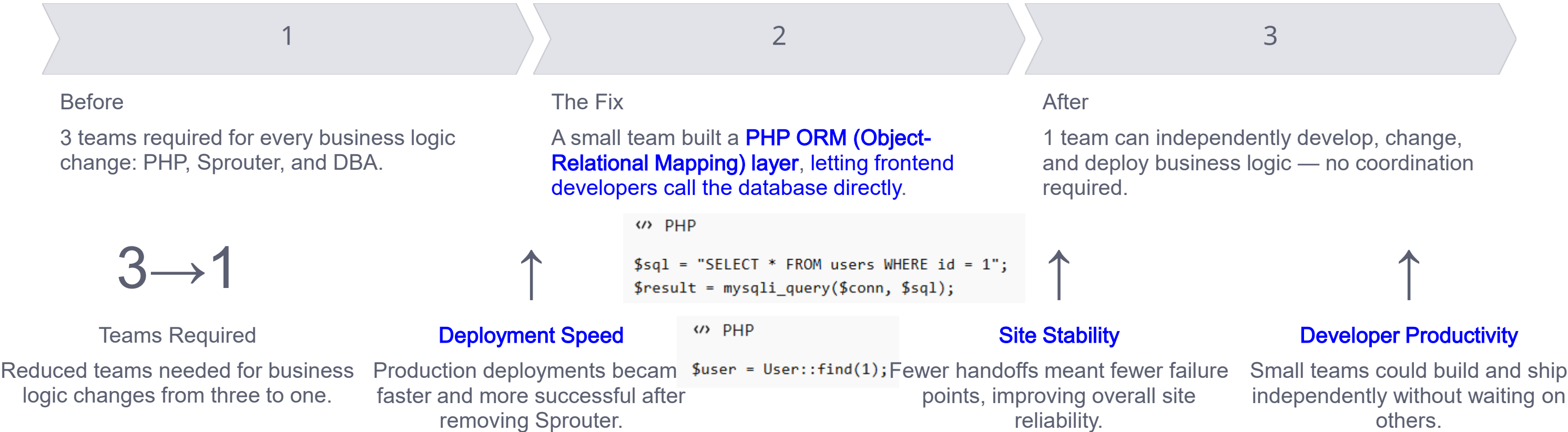
Every time business logic needed to change, developers had to rely on DBAs to write new stored procedures. This created constant back-and-forth, bureaucratic overhead, and friction between teams — slowing down every new feature or update.



Team 1: PHP Developers Wrote application-layer code and business logic requests.	Team 2: Sprouter Maintainers Managed the middleware and routing between app and database.	Team 3: DBAs Wrote and maintained stored procedures in Postgres for every logic change.
--	---	---

Eliminating Sprouter: From Three Teams to One

In 2009, Chad Dickerson joined Etsy as the new CTO as part of a "large-scale cultural transformation." One of his key reform goals: **eliminate Sprouter entirely**. The plan was to move all business logic from the database layer up to the application layer — **reducing the number of teams needed to change business logic from three down to one**.



❏ **Key Takeaway:** By eliminating Sprouter, Etsy removed the coordination overhead of multi-team handoffs — dramatically improving deployment speed, site stability, and developer autonomy. Small teams could now own their code end-to-end.

Organizational Archetypes

How we design our organization determines how work gets done, which in turn affects the results we achieve → Next, **we will explore how to organize our teams so that Conway's Law works to our advantage.**

Functional-Oriented

Optimize for **expertise, division of labor**, or reducing cost. Centralize expertise to enable career growth and skill development with tall hierarchical structures.

This **has been the prevailing method for Operations** (server admins, network admins, database admins **in separate groups**).

Market-Oriented

Optimize for **responding quickly to customer needs**. Tend to be flat, **composed of multiple cross-functional disciplines**.

Used by prominent **DevOps** organizations like Amazon and Netflix, where **each service team is responsible for feature delivery and service support**.

Matrix-Oriented

Attempt to **combine functional and market orientation**. Often result in **complicated structures** with individual contributors reporting to two or more managers.

Sometimes achieve neither the goals of functional nor market orientation.

Problems Often Caused by Overly Functional Orientation

In traditional IT Operations organizations, we often use **functional orientation to organize teams by their specialties**, putting database administrators in one group, network administrators in another, and server administrators in a third.

Long Lead Times

Complex activities like large deployments **require opening tickets with multiple groups** and coordinating work handoffs, **resulting in work waiting in long queues at every step**

Lack of Visibility

Workers **have little understanding of how their work relates to value stream goals**, placing them in a creativity and **motivation gap**.

Competing Priorities

Each function team supports many different projects; they all compete for the same limited resources. As a result, often need managers to decide priorities

Poor Outcomes

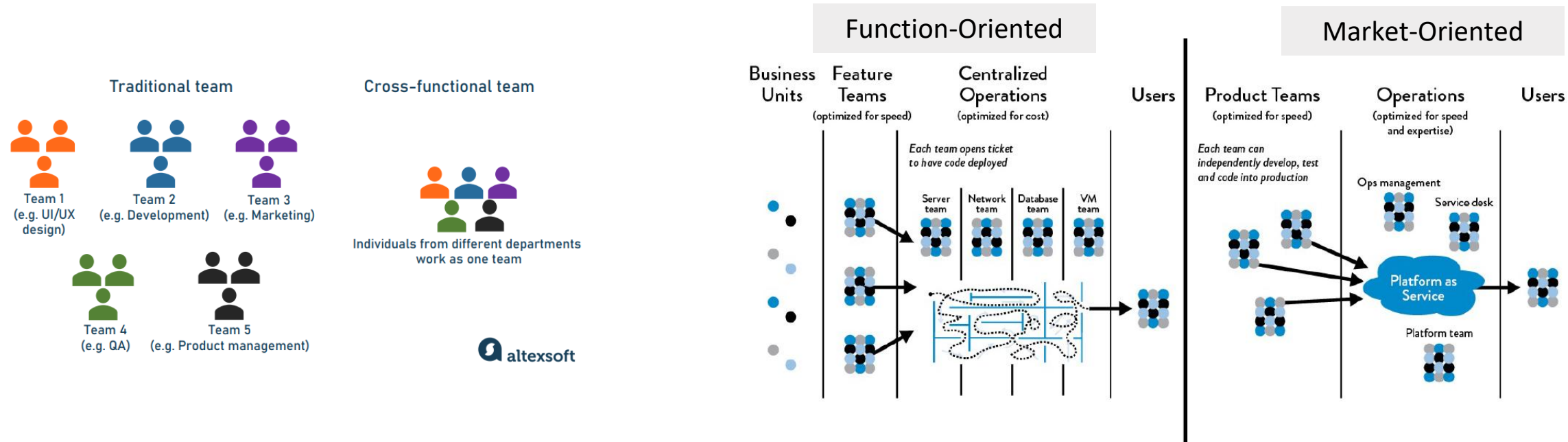
Long queues, poor **handoffs**, large amounts of rework, quality issues, bottlenecks, and **delays** that slow down organizational goals

When every team expedites their work, the net result is that every project ends up moving at the same slow crawl.

* **Lead time is the total time from “when work is requested until the result is delivered”**—whether that’s shipping a product in operations or releasing code in software development. A code change may take 1 minute, but the process around it—tickets, reviews, approvals, and deployment—can take 10 days

Enable Market-Oriented Teams

To achieve DevOps outcomes, we need to reduce the effects of functional orientation and enable market orientation so we can **have many small teams working safely and independently, quickly delivering value to the customer.**



Cross-Functional Teams

Market-oriented teams are responsible for feature development, testing, securing, deploying, and supporting their service in production **from idea conception to retirement**

This model has been adopted by Amazon and Netflix and is claimed as one of the primary reasons behind their ability to move fast even as they grow.

Self-Service Capabilities

Embed functional engineers and skills into each service team, or provide capabilities through automated self-service platforms for environments, testing, and deployments

Independent Delivery

Teams can **independently deliver value to customers without opening tickets** with other groups like IT Operations, QA, or Infosec

Making Functional Orientation Work

Cross-functional and market-oriented teams are one way to achieve fast flow and reliability, but they are not the only path. **Although we have discussed the limitations of functional orientation earlier, we can still achieve DevOps outcomes through functional orientation.**

High performance with a functional-oriented and centralized Operations group is possible, as long as **service teams get what they need from Operations reliably and quickly (ideally on demand) and vice-versa.**

Many of the most admired DevOps organizations retain functional orientation of Operations, including Etsy, Google, and GitHub. What these organizations have in common is a **high-trust culture** that enables all departments to work together effectively.

The Second Toyota Paradox

In the 1980s Lean manufacturing movement, researchers were puzzled by Toyota's functional orientation: **Toyota was organized by functions, not cross-functional teams. Yet it still achieved strong collaboration and continuous improvement.** This shows that structure alone does not create improvement—**how people work and solve problems matters more.**

Designing Team Boundaries with Conway's Law

As organizations grow, maintaining effective communication and coordination becomes one of the greatest challenges. When the primary communication mechanism is **work tickets and change requests** — or worse, when teams are separated by contract boundaries such as outsourced work — **collaboration breaks down**.

As illustrated by the Etsy Sprouter example, how teams are organized creates poor outcomes — a side effect of Conway's Law. These include splitting teams by function (e.g., developers and testers in different locations) or by architectural layer (e.g., application layer, database layer) and sacrifice the trust and collaborations (However, as the Toyota example shows, functional orientation can also be an advantage if there is a culture of trust and fast response mechanisms).

- ❏ **The ideal:** Software architecture should **enable small teams to produce independently**, with enough **decoupling** so that work can proceed without excessive or unnecessary communication and coordination.

Loosely Coupled Architecture Unlocks Developer Productivity

Tightly Coupled Systems

Small changes can cause widespread failures. Teams must constantly coordinate with others who may be affected, navigating complex and bureaucratic change management processes.

Loosely Coupled Systems

Small development teams can independently implement, test, and deploy code to production — safely and quickly. This sustains developer productivity and improves deployment outcomes.

These characteristics are found in Service-Oriented Architecture (SOA), first described in the 1990s, where services can be independently tested and deployed. The core feature of SOA is that it is composed of loosely coupled services and bounded contexts.

Apply Conway's Law in the team design

Keep Teams Small: The "Two-Pizza Rule"

An agile team should be no larger than what two pizzas can feed — typically five to ten people. This reduces inter-team communication and keeps each team's domain scope small but still well-defined.

1

Shared Understanding

Smaller teams maintain a clear, shared understanding of the system. As team size grows, communication needs grow combinatorially.

2

Controlled Growth

Limiting team size making system evolution gradual and manageable rather than chaotic or too fast.

3

Decentralized Power

Each Two-Pizza Team (2PT) operates as autonomously as possible, promoting ownership and independent decision-making.

4

Leadership Pipeline

Leading a 2PT gives employees real leadership experience in an environment where failure is not serious. Amazon links 2PT structure directly to service-oriented architecture.

Understanding Integration and Coupling



Custom Point-to-Point Integration

Every integration between systems must be individually written and designed, tailored to a specific need. For example, if two systems need to exchange data, developers must write custom code to connect them directly. Each connection is unique and purpose-built.

→A better alternative is to use APIs.

Tightly Coupled Systems

The parts of a system are deeply interdependent — changing one part can break others. Any modification requires extreme care, because a single change can ripple across the entire system, disrupting what was previously working.

- ❏ **Key takeaway:** Tightly coupled systems and custom integrations create fragility. Loosely coupled, service-oriented designs are the foundation for teams that can move fast without breaking things.

From Architecture to Value Stream

Architecture Shapes Systems

The structure of a firm directly influences how its systems are designed and built.

Conway's Law

Teamstructure shapes system architecture — your org chart becomes your system diagram.

Poor Boundaries = Slow Teams

Tightly coupled systems emerge from unclear boundaries, creating friction and slowing delivery.

We've learned how structure shapes systems — now we ask: **how does work actually flow from idea to customer value?**

NEW TOOL

Value Stream Mapping

If Conway's Law explains **why systems are structured the way they are**, Value Stream Mapping helps us see **how work actually flows through those systems.**



Main Steps Visible

Reveals each step required to deliver value from idea to customer.



Where Work Waits

Exposes **where work gets stuck, queued**, or blocked in the flow.



Coordination Bottlenecks

Highlights **where handoffs between teams slow things down.**

What Is Value Stream Mapping?

Value Stream Mapping (VSM) is a **flowchart method** used **to illustrate, analyze, and improve the steps required to deliver a product or service** — with a **special focus on identifying and eliminating waste**. Activities are labeled as **"value-adding"** or **"non-value-adding"** from the **customer's perspective**. This approach is closely associated with the **Toyota Production System**.



Continuous Improvement

VSM can be **used repeatedly to drive ongoing improvement** across the entire delivery process.



Spot Waste

Reveals **where resources are wasted** — such as process delays and **bottlenecks** — and helps identify their **root causes**.



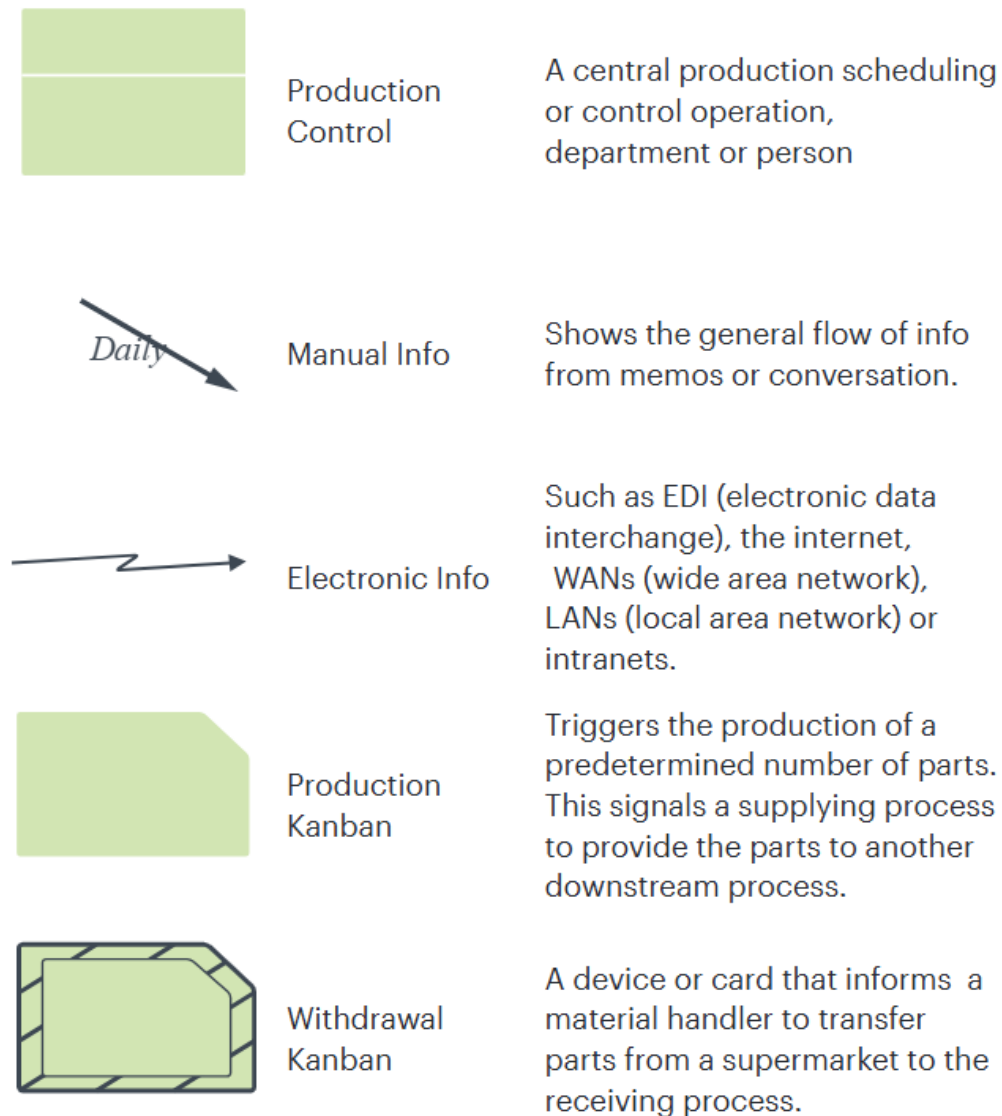
Find Value

Once waste is visible, teams can identify and strengthen the steps that truly add value for the customer.

Like other process diagrams, VSM uses **a standard set of symbols to represent work activities and information flows**. See: lucidchart.com/pages/value-stream-mapping

What Is Value Stream Mapping?

Like other process diagrams, VSM uses **a standard set of symbols to represent work activities and information flows**. See: lucidchart.com/pages/value-stream-mapping



Assembly station needs components



Send a **Kanban card**

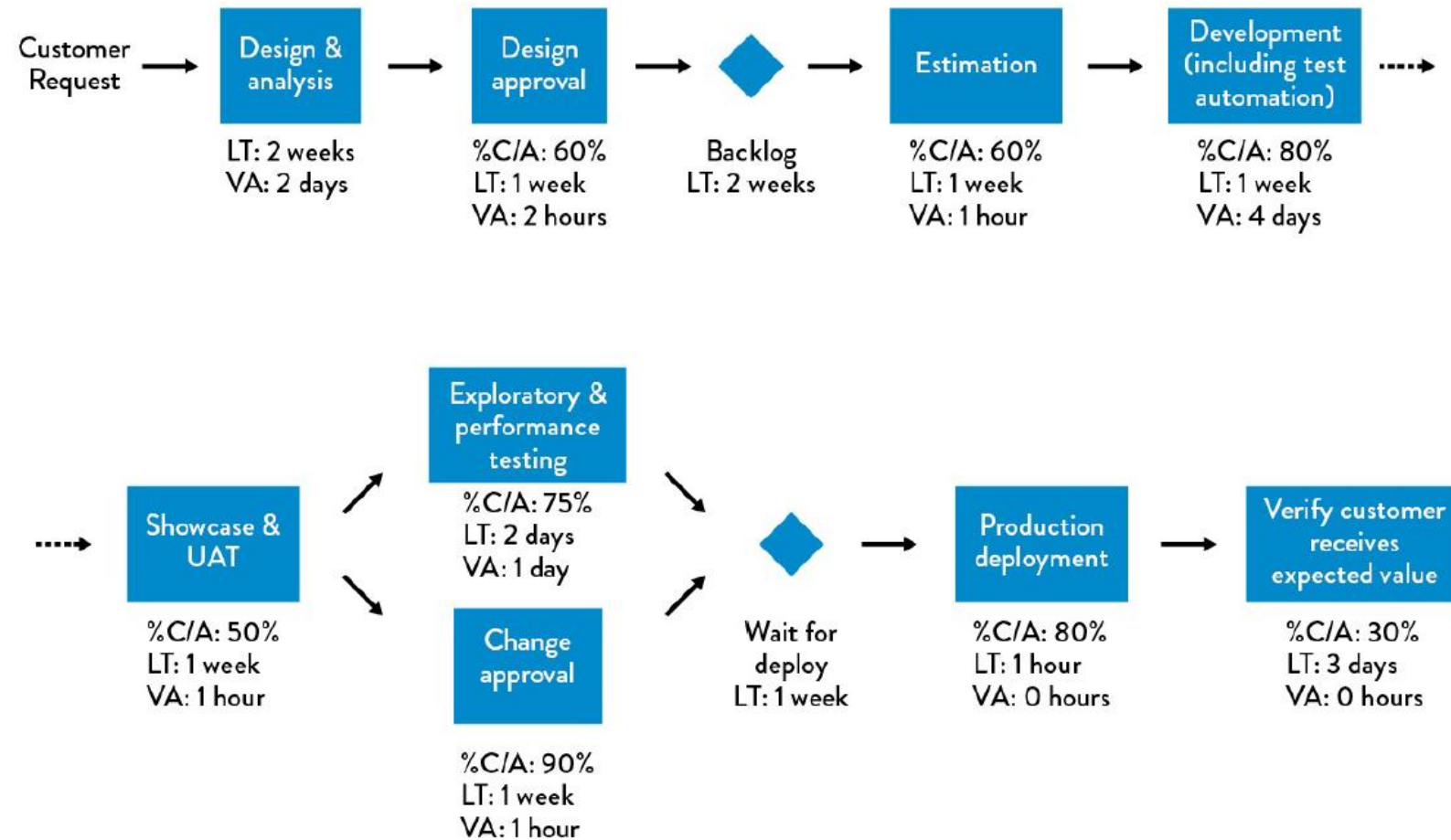


The previous station sees the Kanban and starts production

Therefore, the essence of Kanban is:

“A demand signal card.”

What Is Value Stream Mapping?

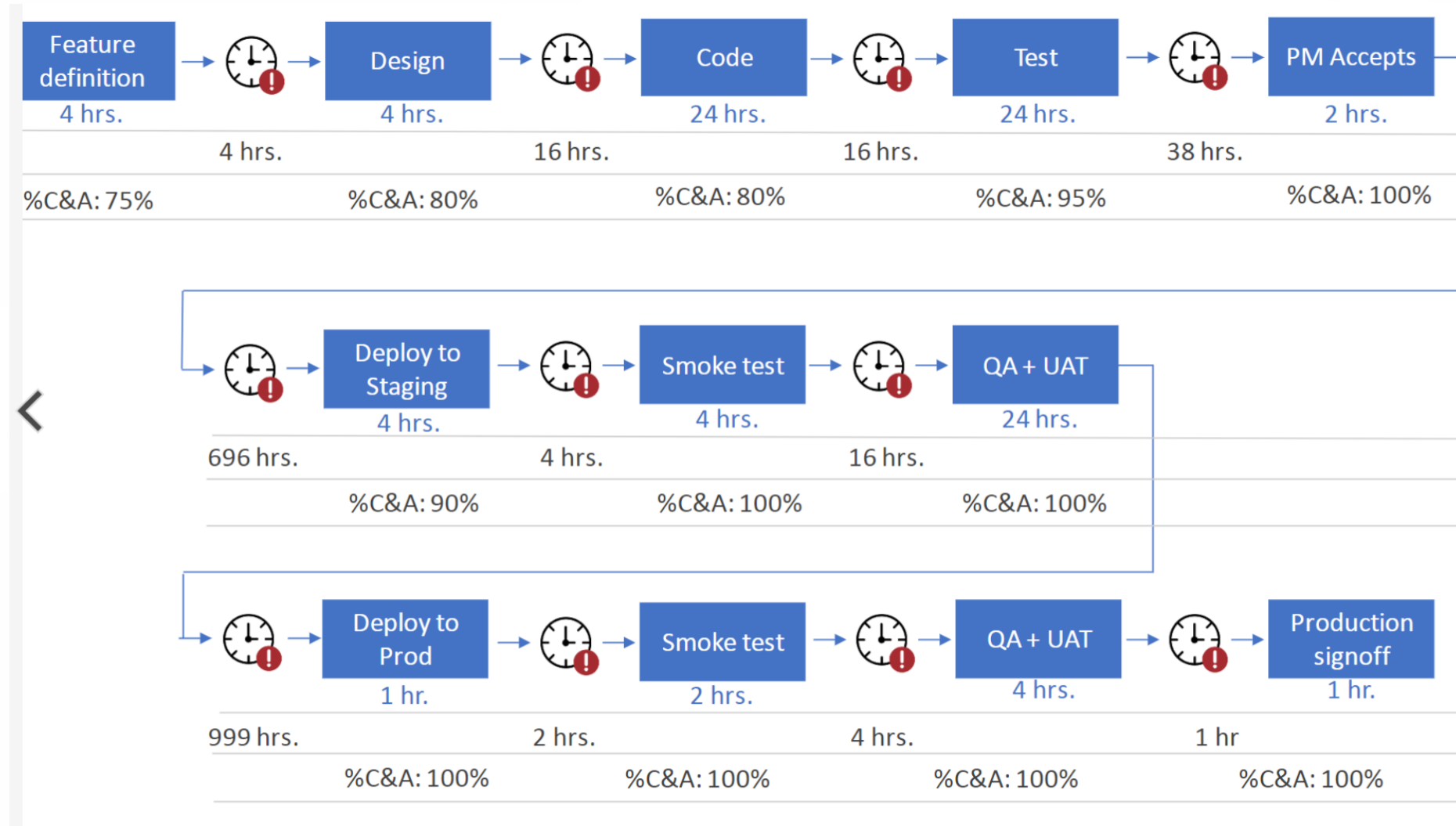
**Aggregate values:**

Total lead time: 10 weeks

Value added time: 7.5 days

Percent complete and accurate: 8.6%

What Is Value Stream Mapping?



(%C/A = Percent Complete and Accurate)

How to Build a Value Stream Map

VSM participants are not limited to a fixed team. In fact, **functionally-oriented organizations** — which often struggle with cross-functional collaboration — **benefit the most.** Members from different departments participate across the upstream and downstream flow of the entire value stream.

The first step is to **identify the value stream members**, then document how work is actually performed. The goal is not to record every detail, but to understand **which nodes block fast flow, short lead times, and reliable outcomes.**

Long Waiting Times

Focus on areas where work waits weeks or months — such as obtaining production-like environments, change approval processes, or security reviews.

High Rework Areas

Identify **where large amounts of rework are generated** or received, as these are key signals of quality and handoff problems.

High-Level Process Blocks

Start with only high-level blocks (**5–15 per map**). Each block should include: **Lead Time, Process Time**, and **%C/A (percent complete and accurate, as measured by downstream consumers)**.

Why Tools Reinforce the Behaviors We Want

In a DevOps value stream, **tools are used to reinforce culture and accelerate behavioral change**. The goal is for tools to help Dev and Ops teams share not just objectives, but **a common work queue in a unified work system with shared vocabulary** — so **work can be prioritized globally rather than in silos** (e.g., Dev using JIRA while Product uses Redmine, Dev using slack while executives using Microsoft teams).

Visible Production Incidents

When incidents appear in the same work system as development tasks, **it becomes obvious when ongoing incidents should stop other work** — especially when using Kanban.

Global Prioritization

A shared work queue lets teams prioritize improvements from an organization-wide view, **selecting work with the highest value** or the greatest reduction in technical debt.

Managing Technical Debt

When technical debt can't be fixed immediately, it's added to the prioritized work queue. Teams can allocate **"20% of time to non-functional requirements"** to systematically address these items.